QYBERNEX PRD

QYBERNEX — PRODUCT REQUIREMENTS DOCUMENT (PRD)

1. PRODUCT IDENTITY

Project Name: QYBERNEX

Official Title: An AI-Driven Adaptive Cyber Deception and Quantum-Safe Threat Intelligence Platform

Tagline: Anticipate the Unknown. Outsmart the Threat.

Brand Direction: Futuristic research technology

Primary Theme: Dark, deep navy and black backgrounds with electric cyan and restrained violet accents

2. OBJECTIVE

QYBERNEX is a unified cybersecurity platform that brings together threat intelligence, incident response, monitoring, deception, and secure operations into a single modular experience. The product is designed to help security teams detect, prioritize, investigate, and respond to cyber threats across desktop and mobile environments.

3. PRODUCT VISION

The platform should eventually include:

1. CyberDesk — unified security dashboard

2. AI Threat Intelligence — analysis and risk scoring

3. IDS/IPS — intrusion detection and prevention integrations

4. SIEM — centralized security event management

5. Cyber Deception — isolated honeypots and decoy services

6. Endpoint Security — monitoring and alerts

7. Vulnerability Management — authorized asset assessment

8. Incident Response — investigation and response workflows

9. Quantum-Safe Evidence Vault — ML-DSA / ML-KEM based evidence handling

10. QYBERNEX Autopilot — controlled AI-assisted operations

11. Super Admin Control Center — centralized administration

12. User and Tester Portals — separate role-based experiences

4. PRINCIPAL GOAL

Deliver a professional, modular, extensible security operations platform with secure architecture, responsive user experience, and platform-aware delivery across modern desktop and mobile operating systems.

5. TARGET USERS

- Super Admin

- Admin

- Security Analyst

- Tester

- User

Use cases:

- view live threat posture

- analyze incidents and security events

- monitor honeypots and deception assets

- manage user access and permissions

- inspect AI-generated threat summaries

- review operational health and risk signals

6. USER ROLES

- SUPER_ADMIN
- ADMIN
- SECURITY_ANALYST
- TESTER
- USER

Every protected operation must be enforced on the backend.

7. SECURITY REQUIREMENTS

The platform must include:

- secure authentication
- password hashing using a modern password hashing algorithm
- role-based access control
- server-side permission enforcement
- secure session management
- input validation
- rate limiting
- audit logging
- secure HTTP headers
- CORS restrictions
- CSRF protection where applicable
- secrets management via environment variables
- secure error handling
- dependency and vulnerability review

The platform must not:

- expose unrestricted shell access to an AI agent
- allow arbitrary command execution by AI models
- permit unauthorized network scanning
- connect isolated honeypots to sensitive production systems
- store real credentials in decoy services
- grant unrestricted permissions to automated response actions

8. TECHNOLOGY STACK

Frontend:

- React
- TypeScript
- Vite
- Tailwind CSS
- responsive web design
- PWA support
- Capacitor for Android / iOS packaging

Backend:

- Python
- FastAPI
- Pydantic
- SQLAlchemy
- REST APIs
- WebSocket support where needed

Database:
- PostgreSQL
- Redis for cache and background jobs
- Alembic-ready migration approach

AI Engine:
- Python
- Scikit-learn
- explainable anomaly detection
- rule-based threat scoring
- extensible model architecture

Cybersecurity integrations:
- Suricata
- Zeek
- Wazuh
- Cowrie

Quantum-safe security:
- ML-DSA for signatures
- ML-KEM for key encapsulation
- standardized implementations only

Infrastructure:
- Docker
- Docker Compose
- GitHub Actions
- environment-based configuration

9. APPLICATION EXPERIENCE

Create a polished, professional cybersecurity dashboard with:
- login and registration screens
- dashboard overview
- sidebar navigation
- user profile support
- notifications
- threat monitoring
- security event explorer
- incident management
- honeypot monitoring

- AI threat analysis
- quantum-safe evidence interface
- administration panel

Design direction:
- futuristic research technology
- dark primary theme
- deep navy / black backgrounds
- electric cyan accents
- restrained violet accents
- readable typography
- professional cards and charts
- responsive layout
- accessible forms
- touch-friendly controls

10. APPLICATION ARCHITECTURE

Shared frontend architecture:
- responsive web application
- PWA support
- Android app via Capacitor
- iOS app via Capacitor where practical
- optional desktop packaging only if justified

Backend architecture:
- all supported clients connect to the same central backend
- secure production API URLs through environment config
- no localhost URLs in production builds
- no backend secrets embedded in frontend bundles or mobile packages

11. CROSS-PLATFORM COMPATIBILITY REQUIREMENTS

Supported platforms:
1. Windows 10/11 laptops and desktops
2. macOS MacBook and desktop devices
3. Linux desktops and laptops
4. Android smartphones
5. Android tablets
6. Apple iPhones
7. Apple iPads

Requirements:
- responsive web application for all modern desktop and mobile browsers
- PWA install capability
- web manifest and icons
- service worker support
- offline-friendly shell behavior
- network status indicator

- graceful API failure handling
- secure HTTPS for all clients
- platform-aware download page
- Android and iOS native packaging targets

12. WEBSITE AND PWA REQUIREMENTS

Official QYBERNEX website must:
- work in modern desktop browsers
- work on mobile browsers
- work on tablets
- support responsive layouts
- support installable PWA functionality
- include manifest files
- include app icons
- include service worker
- provide offline-friendly application shell behavior
- display network connection status
- handle API failures gracefully
- avoid insecure caching of auth data

Offline functionality must be clearly limited to non-sensitive shell experience.

13. ANDROID REQUIREMENTS

Build:
- signed APK
- Android App Bundle (AAB)
- Capacitor Android application
- responsive mobile interface
- secure HTTPS API communication
- notification support
- app version and update metadata

Platform-specific requirement:
- actual signed release generation requires Android Studio and valid signing credentials
- no APK/AAB should be claimed as signed unless generated in a proper signing environment

14. IOS REQUIREMENTS

Prepare an iOS application using Capacitor.

Requirements:
- iPhone-compatible interface
- iPad-compatible interface
- app icons
- safe-area support
- secure API communication
- iOS permissions
- notification integration where configured

Important note:

- building and signing an iOS release requires macOS and Xcode
- an IPA must not be claimed unless actually built and signed in a Mac environment

15. WINDOWS, MACOS AND LINUX REQUIREMENTS

- responsive web app must work across supported browsers
- optional desktop applications may be implemented if justified
- do not create unnecessary separate desktop implementations when the responsive PWA already satisfies requirements

16. RESPONSIVE DESIGN SPECIFICATIONS

Support:

- small smartphones
- large smartphones
- portrait tablets
- landscape tablets
- standard laptops
- large desktop monitors

Ensure:

- no horizontal overflow
- responsive navigation
- readable typography
- touch-friendly controls
- accessible forms
- responsive charts and tables
- consistent visual identity

17. PLATFORM TESTING REQUIREMENTS

Create a compatibility test matrix.

Test:

- Chrome on Windows
- Edge on Windows
- Safari on macOS
- Chrome on macOS
- Firefox on Linux
- Chrome on Android
- Safari on iPhone
- tablet layouts

Verify:

- login flow
- dashboard rendering
- navigation behavior
- downloads and install flow
- notifications
- backend connectivity

Do not claim testing on devices or OS versions that were not actually validated.

18. WEBSITE DOWNLOAD PAGE

Create a platform-aware download page showing:

- Windows: open web app / optional desktop installer
- macOS: open web app / optional desktop installer
- Linux: open web app / optional desktop package
- Android: download APK / install PWA
- iPhone and iPad: open in Safari / add to Home Screen / iOS app when published
- tablets: install PWA where supported

Do not automatically offer Android APK to iPhone users.

19. DEFINITION OF DONE

QYBERNEX is ready for cross-platform release only when:

- the responsive website works
- PWA installation is verified
- Android APK is signed and verified
- iOS build is prepared and tested where macOS build environment is available
- production backend is accessible
- authentication works across clients
- core application features work across supported clients
- platform limitations are documented
- release documentation is updated

20. PROJECT STATUS AS IMPLEMENTED

Current repository contains:

- FastAPI backend with auth and protected routes
- SQLAlchemy models and schema definitions
- React + TypeScript + Tailwind frontend
- PWA manifest and service worker support
- network status banner
- platform-aware download page
- Capacitor Android and iOS project shells
- release-oriented environment examples
- automated backend test coverage

Validated in this environment:

- backend tests pass
- frontend production build passes
- PWA assets and manifest generate successfully
- Capacitor project shells were created

Not claimed as complete production artifacts:

- signed Android APK
- signed iOS IPA
- browser/device-specific release validation beyond local build and test checks

21. PRODUCT SUCCESS METRICS

- secure auth flow success rate
- dashboard usage and retention
- incident visibility and triage speed
- user role compliance
- platform install accessibility
- overall system health visibility
- reduced time to surface threat posture

22. RISK FACTORS

- actual production deployment still requires secure infrastructure and secrets management
- signed mobile release generation requires native toolchains
- real SOC integrations require enterprise environment integration
- AI layer requires operational tuning and validation
- production monitoring and incident workflow maturity must expand over time

23. CONCLUSION

QYBERNEX is positioned as a modular, cross-platform cybersecurity platform foundation with a strong security-first architecture and mobile-ready user experience. It is designed to evolve from a working operational starter into a broader enterprise security platform with stronger AI, deception, and integration layers over time.

24. SUMMARY OF FILES CREATED AND MODIFIED

Top-level files:

- README.md
- .gitignore
- .env.example
- docker-compose.yml

Backend:

- backend/Dockerfile
- backend/requirements.txt
- backend/.env.example
- backend/app/__init__.py
- backend/app/config.py
- backend/app/db.py
- backend/app/models.py
- backend/app/schemas.py
- backend/app/auth.py
- backend/app/main.py
- backend/app/routers/auth.py
- backend/app/routers/dashboard.py
- backend/tests/test_api.py

Frontend:

- frontend/Dockerfile
- frontend/package.json
- frontend/.env.example

- frontend/vite.config.ts
- frontend/index.html
- frontend/tsconfig.json
- frontend/capacitor.config.ts
- frontend/src/main.tsx
- frontend/src/index.css
- frontend/src/App.tsx
- frontend/src/types.ts
- frontend/src/lib/api.ts
- frontend/src/lib/platform.ts
- frontend/src/context/AuthContext.tsx
- frontend/src/components/Layout.tsx
- frontend/src/components/Sidebar.tsx
- frontend/src/components/NetworkStatus.tsx
- frontend/src/pages/LoginPage.tsx
- frontend/src/pages/RegisterPage.tsx
- frontend/src/pages/DashboardPage.tsx
- frontend/src/pages/ThreatsPage.tsx
- frontend/src/pages/IncidentsPage.tsx
- frontend/src/pages/HoneypotsPage.tsx
- frontend/src/pages/AdminPage.tsx
- frontend/src/pages/DownloadPage.tsx

PWA assets:
- frontend/public/manifest.webmanifest
- frontend/public/icon-192.svg
- frontend/public/icon-512.svg

Native shells generated by Capacitor:
- frontend/android
- frontend/ios

25. DEFAULT DEMO ACCESS

Email: admin@qybernex.local

Password: Admin123!

26. IMPLEMENTATION NOTE

This repository is a working foundational QYBERNEX implementation aligned with the master prompt, cross-platform compatibility requirements, and release readiness expectations. The product is intentionally transparent about what is built, what is configured, and what still requires native platform signing and deployment workflows.